HONEYWELL SOFTWARE ENGINEERING TEAMS

AI Champions Programme

Module 03

Prompt Engineering Recap and Context Engineering

A practical recap of prompt engineering, then a move into selecting, structuring, layering, and compressing the context supplied to AI — without letting token usage inflate.

DURATION
2 Hours

FORMAT
Hands-On Lab

DELIVERED FOR
Honeywell Eng. Teams

How We'll Spend These Two Hours

Six connected moves — from a prompting recap to a hands-on context-engineering comparison.

01

Prompt Engineering Recap: The Six Moves

15 MIN

Task framing, role/instruction, constraints, examples, decomposition, and iterative verification.

02

From Prompt-Only to Context-Engineered

10 MIN

Why structured prompting alone still isn't enough, and what context engineering adds.

03

The Context Sources Map

15 MIN

Repository files, design notes, APIs, standards, issue history, specifications and tests.

04

The Context Engineering Architecture & Layering

25 MIN

How raw context is selected, structured, compressed, and scoped before it ever reaches the agent.

05

Context Boundaries: Avoiding Token Inflation

20 MIN

Token-aware practices and the habits that quietly inflate context without adding value.

06

Hands-On Lab: Prompt-Only vs Context-Engineered

35 MIN

Solve the same task both ways and compare quality, token usage, iterations, and time.

Module 03 in the 12-Module Journey

Context discipline here is what Module 04's specifications and every later module's token economics depend on.

Why this matters: Spec-Driven Development in Module 04 is only as precise as the context behind it — and every later module is measured in part on token usage and AI cost, both set by the habits this session builds.

Prompt Engineering Recap: The Six Moves

The practical sequence behind any well-formed prompt, before context engineering is added on top.

Frame the Task	Set Role & Instruction	Add Constraints	Give Examples	Decompose	Iterate & Verify
State the goal in one clear sentence — what should exist when the task is done.	Tell the model what perspective to take and how to approach the task.	Name the limits — language, style, performance, or interfaces that must be respected.	Show, don't just tell — a short example anchors format and quality expectations.	Break a large task into smaller, independently verifiable steps.	Check the output against the goal, and refine the prompt rather than accepting the first pass.

The throughline: These six moves are necessary but not sufficient — they shape how you ask, but say nothing about what the model actually knows about your repository. That's what context engineering adds next.

From Prompt-Only to Context-Engineered

Three maturity stages — this module moves the room from left to right.

1

Prompt-Only

A single well-worded prompt, with no repository context attached. Works for small, self-contained tasks; breaks down on anything that touches existing code.

2

Structured Prompting

The six recap moves applied consistently — role, constraints, examples, decomposition. Better and more repeatable, but still no persistent view of the repository.

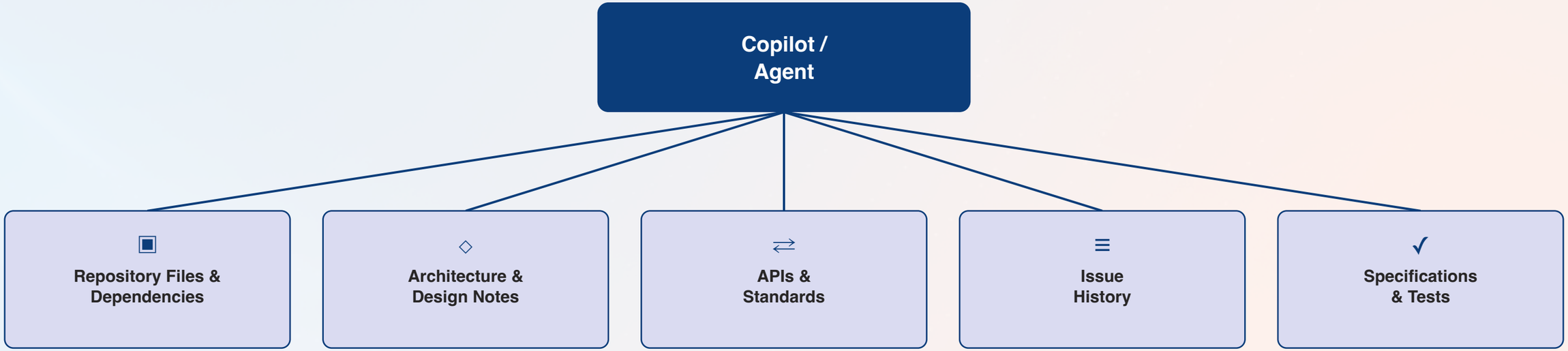
3

Context-Engineered

Repository, design, API, and spec context deliberately selected, layered, compressed, and scoped to the task before the prompt is ever sent.

The Context Sources Map

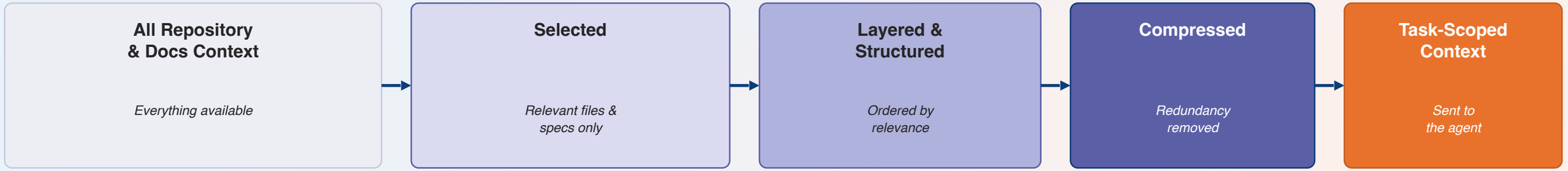
Everything a task's context could draw from — before any selection happens.



Why this matters: Every one of these sources is a candidate for context — the discipline this module teaches is choosing which of them actually belong in a given task's prompt.

The Context Engineering Pipeline

Raw sources narrow, stage by stage, into exactly what the task needs — and nothing more.



The throughline: Selection, layering, and compression aren't optional polish — skipping them is exactly how context inflation and avoidable token cost happen at scale.

Context Boundaries: What Actually Reaches the Agent

Everything outside the innermost box is available, but deliberately not sent.

Why this matters: Drawing this boundary deliberately, task by task, is what keeps token usage predictable — the opposite of letting context grow by default.

Anti-Patterns: How Context Inflation Happens

The habits that quietly grow token usage without improving output quality.

DO THIS	NOT THAT
Select only the files/snippets the task actually touches	Attach whole files when only a few functions are relevant
Reuse or reference prior context instead of resending it	Re-send the same context on every follow-up turn
Summarize specs and issues to the parts that matter	Copy entire specs or issue threads verbatim into the prompt
Reset or scope context when the task changes	Let context accumulate across an unrelated multi-task session
Compress deliberately — a smaller, sharper context outperforms a bigger, vaguer one	Skip compression because the model "has a big context window"

Hands-On Lab Preview: Prompt-Only vs Context-Engineered

The 35-minute activity that closes this module — the same task, worked both ways.

Measure	Prompt-Only	Context-Engineered
Output quality	Assessed as-is, first pass	Assessed against the same rubric
Token usage	Logged from the worksheet	Logged from the worksheet
Iteration count	Counted to reach an acceptable result	Counted to reach an acceptable result
Task completion time	Timed end to end	Timed end to end

Both passes use the token usage worksheet — this module's baseline for every later module's cost discussion.

Module 03 Outcomes & What's Next

- ✓ The six practical prompt engineering moves are refreshed and applied
- ✓ The gap between structured prompting and context engineering is clear
- ✓ The full context sources map is understood
- ✓ The select → layer → compress → scope pipeline is understood
- ✓ Context boundaries and anti-patterns for token inflation are identified
- ✓ Prompt-only and context-engineered approaches have been compared on real metrics

NEXT MODULE

Module 04 — Spec-Driven Development with GitHub Spec Kit / OpenSpec (2 hours). Using specification, architecture/plan, tasks, and acceptance criteria to drive traceable greenfield implementation.

Questions & Discussion

Module 03 — Prompt Engineering Recap and Context Engineering